

# SyncBatchNorm#

<https://pytorch.org/docs/stable/generated/torch.nn.SyncBatchNorm.html#torch.nn.SyncBatchNorm>

## Description:

Applies Batch Normalization over a N-Dimensional input. The N-D input is a mini-batch of  $[N-2]D$  inputs with additional channel dimension) as described in the paper Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift . The mean and standard-deviation are calculated per-dimension over all mini-batches of the same process groups.  $\gamma$  and  $\beta$  are learnable parameter vectors of size  $C$  (where  $C$  is the input size). By default, the elements of  $\gamma$  are sampled from  $U(0,1)$  and the elements of  $\beta$  are set to 0. The standard-deviation is calculated via the biased estimator, equivalent to `torch.var(input, unbiased=False)`. Also by default, during training this layer keeps running estimates of its computed mean and variance, which are then used for normalization during evaluation. The running estimates are kept with a default momentum of 0.1. If `track_running_stats` is set to `False`, this layer then does not keep running estimates, and batch statistics are instead used during evaluation time as well.

## Function Signature

---

```
class torch.nn.SyncBatchNorm(num_features, eps=1e-05,
momentum=0.1, affine=True, track_running_stats=True,
process_group=None, device=None, dtype=None)[source]#
```

Parameters

Shape:

```
classmethod convert_sync_batchnorm(module,
process_group=None)[source]#
```

Parameters

**Returns:**

Tensor

## Code Examples

---

```

>>> # With Learnable Parameters
>>> m = nn.SyncBatchNorm(100)
>>> # creating process group (optional)
>>> # ranks is a list of int identifying rank ids.
>>> ranks = list(range(8))
>>> r1, r2 = ranks[:4], ranks[4:]
>>> # Note: every rank calls into new_group for every
>>> # process group created, even if that rank is not
>>> # part of the group.
>>> process_groups = [torch.distributed.new_group(pids) for pids
in [r1, r2]]
>>> process_group = process_groups[0 if dist.get_rank() <= 3
else 1]
>>> # Without Learnable Parameters
>>> m = nn.BatchNorm3d(100, affine=False,
process_group=process_group)
>>> input = torch.randn(20, 100, 35, 45, 10)
>>> output = m(input)

>>> # network is nn.BatchNorm layer
>>> sync_bn_network =
nn.SyncBatchNorm.convert_sync_batchnorm(network, process_group)
>>> # only single gpu per process is currently supported
>>> ddp_sync_bn_network =
torch.nn.parallel.DistributedDataParallel(
>>>         sync_bn_network,
>>>         device_ids=[args.local_rank],
>>>         output_device=args.local_rank)

```

```
>>> # Network with nn.BatchNorm layer
>>> module = torch.nn.Sequential(
>>>     torch.nn.Linear(20, 100),
>>>     torch.nn.BatchNorm1d(100),
>>> ).cuda()
>>> # creating process group (optional)
>>> # ranks is a list of int identifying rank ids.
>>> ranks = list(range(8))
>>> r1, r2 = ranks[:4], ranks[4:]
>>> # Note: every rank calls into new_group for every
>>> # process group created, even if that rank is not
>>> # part of the group.
>>> process_groups = [torch.distributed.new_group(pids) for pids
in [r1, r2]]
>>> process_group = process_groups[0 if dist.get_rank() <= 3
else 1]
>>> sync_bn_module =
torch.nn.SyncBatchNorm.convert_sync_batchnorm(module,
process_group)
```

## Notes & Warnings

---

### NOTE

Note This momentum argument is different from one used in optimizer classes and the conventional notion of momentum. Mathematically, the update rule for running statistics here is  $x^{\text{new}} = (1 - \text{momentum}) \times x^{\text{old}} + \text{momentum} \times x_t$ , where  $x^{\text{old}}$  is the estimated statistic and  $x_t$  is the new observed value.

### NOTE

Note Synchronization of batchnorm statistics occurs only while training, i.e. synchronization is disabled when `model.eval()` is set or if `self.training` is otherwise False.

## Detailed Documentation

---

### Documentation

Applies Batch Normalization over a N-Dimensional input.

The N-D input is a mini-batch of [N-2]D inputs with additional channel dimension) as described in the paper Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift .

The mean and standard-deviation are calculated per-dimension over all mini-batches of

the same process groups.  $\gamma$  and  $\beta$  are learnable parameter vectors of size  $C$  (where  $C$  is the input size). By default, the elements of  $\gamma$  are sampled from  $U(0,1)$  and the elements of  $\beta$  are set to 0. The standard-deviation is calculated via the biased estimator, equivalent to `torch.var(input, unbiased=False)`.

Also by default, during training this layer keeps running estimates of its computed mean and variance, which are then used for normalization during evaluation. The running estimates are kept with a default momentum of 0.1.

If `track_running_stats` is set to `False`, this layer then does not keep running estimates, and batch statistics are instead used during evaluation time as well.

This momentum argument is different from one used in optimizer classes and the conventional notion of momentum. Mathematically, the update rule for running statistics here is  $x^{\text{new}} = (1 - \text{momentum}) \times x^{\text{old}} + \text{momentum} \times x_{\text{new}}$ .  

$$x^{\text{new}} = (1 - \text{momentum}) \times x^{\text{old}} + \text{momentum} \times x_{\text{new}}$$
where  $x^{\text{old}}$  is the estimated statistic and  $x_{\text{new}}$  is the new observed value.

Because the Batch Normalization is done for each channel in the  $C$  dimension, computing statistics on  $(N, +)$  slices, it's common terminology to call this Volumetric Batch Normalization or Spatio-temporal Batch Normalization.

Currently `SyncBatchNorm` only supports `DistributedDataParallel (DDP)` with single GPU per process. Use `torch.nn.SyncBatchNorm.convert_sync_batchnorm()` to convert `BatchNorm*D` layer to `SyncBatchNorm` before wrapping Network with DDP.

`num_features (int)` – CCC from an expected input of size  $(N,C,+)$

`eps (float)` – a value added to the denominator for numerical stability. Default:  $1e-5$

momentum (Optional[float]) – the value used for the running\_mean and running\_var computation. Can be set to None for cumulative moving average (i.e. simple average). Default: 0.1

affine (bool) – a boolean value that when set to True, this module has learnable affine parameters. Default: True

track\_running\_stats (bool) – a boolean value that when set to True, this module tracks the running mean and variance, and when set to False, this module does not track such statistics, and initializes statistics buffers running\_mean and running\_var as None. When these buffers are None, this module always uses batch statistics. in both training and eval modes. Default: True

process\_group (Optional[Any]) – synchronization of stats happen within each process group individually. Default behavior is synchronization across the whole world

Input: (N,C,+)(N, C, +)(N,C,+)

Output: (N,C,+)(N, C, +)(N,C,+) (same shape as input)

Synchronization of batchnorm statistics occurs only while training, i.e. synchronization is disabled when model.eval() is set or if self.training is otherwise False.

Converts all BatchNorm\*D layers in the model to torch.nn.SyncBatchNorm layers.

module (nn.Module) – module containing one or more BatchNorm\*D layers

process\_group (optional) – process group to scope synchronization, default is the whole world

The original module with the converted torch.nn.SyncBatchNorm layers. If the original module is a BatchNorm\*D layer, a new torch.nn.SyncBatchNorm layer object will be

returned instead.

Runs the forward pass.